

Session 08 Teamwork — Validate the Workspaces Feature

Context

Work in your team (six teams of four students). The instructor's own `ProjectsModule` now has real DTOs, a global `ValidationPipe`, and a brand-new validated `POST /projects` endpoint. Your job is to build the exact same shape of validation, independently, for a second B4F Board resource: Workspaces.

Goal

`WorkspacesController / WorkspacesService` validated with the same DTO pattern as `Projects` — including a brand-new `POST /workspaces` — reasoned through by your team, not copied and renamed from `projects/dto/*.ts`.

What to build

- `UpdateWorkspaceDto` (in `src/workspaces/dto/update-workspace.dto.ts`): `name?` and `slug?`, each `@IsOptional()` `@IsString()` `@IsNotEmpty()` — the same three decorators as `UpdateProjectDto`, on `Workspaces`' own real fields.
- `CreateWorkspaceDto` (in `src/workspaces/dto/create-workspace.dto.ts`): `name` and `slug`, each `@IsString()` `@IsNotEmpty()` — required, no `@IsOptional()`.
- `WorkspacesService.create(dto)`: 409 if the new `slug` is already used by another workspace, otherwise create and return a new workspace with a real unique `id`.
- `WorkspacesService.update(id, dto)`: drop the old manual empty-name check (now redundant with the DTO's own `@IsNotEmpty()`) — keep the `slug`-uniqueness check exactly as it is, since no decorator could express it.
- `WorkspacesController`: add `@Post()` wired to `create()`; change `update()`'s `@Body()` parameter from the old inline object type to `UpdateWorkspaceDto`.

Rules

Allowed: everything the instructor just used for `Projects` — `@IsOptional()`, `@IsString()`, `@IsNotEmpty()`, `@Post()`, and the global `ValidationPipe` already wired once for the whole application (nothing to change in `main.ts`).

Not yet: no new `ValidationPipe` options, no guards, no database, no new npm package, no `description` field (that is next session's own homework).

Test it

BASH

```
curl http://localhost:3000/workspaces
curl -X POST http://localhost:3000/workspaces -H "Content-Type: application/json" -d
'{"name": "Blue Ridge", "slug": "blue-ridge"}'
curl -X POST http://localhost:3000/workspaces -H "Content-Type: application/json" -d
'{"name": ""}'
curl -X PATCH http://localhost:3000/workspaces/1 -H "Content-Type: application/json" -d
'{"name": ""}'
curl -X PATCH http://localhost:3000/workspaces/1 -H "Content-Type: application/json" -d
'{"slug": "acme-robotics"}'
```

Done when

- `POST /workspaces` with a valid body returns `201` and a new workspace with a real new id.
- `POST /workspaces` with a missing/empty `name` returns `400`, generated automatically — no `if` statement written for it.
- `PATCH /workspaces/:id` still returns `400` for an empty name (now from the DTO, not a hand-written check) and `409` for a `slug` collision (still a hand-written check — on purpose).
- Every team member can point at the one check in `update()` that is NOT a DTO decorator, and explain why it couldn't be one.

Working time

Around 25 minutes.